Beyond Code Smell:

Measuring Architectural Conformance Drift in Long-Horizon Autonomous Coding Agent Sessions

Vikhram S
Empirical Software Engineering Preprint Under Peer Review

Abstract

Autonomous coding agents (e.g., Claude Code, OpenHands, SWE-agent) increasingly execute multi-session, long-horizon software engineering tasks across multi-file codebases. While existing benchmarks measure task resolution rates (pass/fail tests) and localized code quality erosion (e.g., SlopCodeBench), they fail to evaluate whether autonomous agents preserve higher-level architectural models, module boundaries, and layering constraints over extended operational horizons.

In this empirical study, we introduce an automated, fault-tolerant measurement instrument that extracts static dependency graphs $G = (V, E)$ and evaluates explicit architectural fitness functions across 18 open-source repositories, stratified across language, size, and domain.

We formalize four quantitative drift metrics: Dependency Graph Edit Distance (DGED), Layering Violation Count (LVC), Conformance Decay Rate (β_{CDR}), and Complexity-Normalized Layering Violation (CNLV).

Comparing agent session trajectories against historical human commit trajectories, initial preliminary findings [`DRY-RUN / PRELIMINARY DATASET`] indicate that agent sessions introduce statistically significant architectural drift ($U = 2189.5$, $p = 1.19 \times 10^{-6}$, $r_{\text{rb}} = -0.502$), and that architectural drift strongly predicts downstream test suite failure ($r_s = +0.400$, $p = 0.0027$, Odds Ratio $= 2.07 \times 10^4$).

We provide a fully reproducible, zero-GPU evaluation harness and open dataset.

Keywords: Autonomous Coding Agents, Software Architecture, Conformance Drift, Static Analysis, Empirical Software Engineering, Long-Horizon Agents

1 Introduction

Autonomous LLM-driven coding agents are rapidly advancing from single-file bug fixes to long-horizon, multi-session software engineering workflows. Scaffolds such as Claude Code, OpenHands, and SWE-agent operate across sequential sessions to implement complex feature requests, refactor legacy packages, and resolve integration issues.

However, while an agent may generate syntactically valid code that satisfies localized unit tests, it frequently introduces subtle, structural architectural violations—such as bypassing abstractions, introducing circular package dependencies, or leaking domain entities into infrastructure layers.

This phenomenon, which we term *Architectural Conformance Drift*, represents a fundamental threat to long-term software maintainability. Unlike localized "code smell" or readability erosion, architectural drift accumulates silently over multi-session agent invocations. When human engineers attempt to maintain codebases heavily modified by autonomous agents, they inherit structurally compromised systems where layer boundaries have dissolved.

While recent benchmarks evaluate agent performance on single context windows (SWE-bench) or code-quality degradation (SlopCodeBench, RoadmapBench), none explicitly measure whether an agent's structural edits adhere to an explicit, machine-checkable architectural model.

To address this gap, this paper asks the following core research questions:

RQ1 Decay Trajectory: As autonomous coding agents work across multi-step, multi-session tasks, does the codebase's architecture drift measurably away from an explicit architectural model over time?

RQ2 Human Baseline Comparison: How does agent-induced architectural drift differ in magnitude and velocity from human-authored commit histories on the exact same repositories?

RQ3 Practical Utility & Risk: Does higher architectural conformance drift correlate with downstream test suite failure or task regression rates?

2 Related Work

Our work sits at the intersection of three active bodies of research in empirical software engineering.

2.1 Long-Horizon Agent Benchmarks

Existing benchmarks primarily evaluate task resolution pass rates on isolated context windows (e.g., SWE-bench, HumanEval). Recent extensions examine multi-step task execution, such as RoadmapBench and SlopCodeBench (arXiv:2603.24755).

However, these benchmarks focus on code smell erosion (cyclomatic complexity, line length, duplication) rather than explicit macro-architectural conformance.

2.2 Architecture Conformance & Fitness Functions

Static analysis tools (e.g., `pydeps`, `grimp`, `dependency-cruiser`) extract module dependency graphs to enforce layering rules and fitness functions.

Classical empirical studies demonstrate that human architectural violations increase technical debt. We adapt these classical static analysis instruments to autonomous agent sessions.

2.3 Agentic Software Engineering Frameworks & Entropy

Recent studies explore "agentic entropy" and "Beyond the Diff" paradigm shifts. We differentiate our work by providing a formal, graph-theoretic formulation of architectural conformance drift (CNLV) evaluated against human commit baselines under strict API cost constraints.

3 Methodology

3.1 Repository Corpus & Stratified Sampling

We selected 18 open-source repositories across Python and TypeScript, stratified evenly across size tiers (*Small*, *Medium*, *Large*) and application domains (*Web Backend*, *CLI Tool*, *Data Library*).

Repositories were required to have permissive licenses (MIT/Apache/BSD), established test suites, and clear layer boundaries.

3.2 Architectural Ground-Truth & Metric Formalization

For each repository, we extract a baseline import Directed Acyclic Graph

$$G_0 = (V_0, E_0) \tag{1}$$

using `grimp` and compute baseline Cyclomatic Complexity CC_0.

Explicit layering rules are encoded as forbidden dependency edges

$$E_{\text{forbidden}} \subset V \times V. \tag{2}$$

After each agent session $t \in \{1, 2, 3\}$, we extract

$$G_t = (V_t, E_t) \tag{3}$$

and compute the following metrics.

3.2.1 Dependency Graph Edit Distance (DGED)

$$\boxed{\mathrm{DGED}(G_0, G_t) = |V_0 \Delta V_t| + |E_0 \Delta E_t|} \tag{4}$$

3.2.2 Layering Violation Count (LVC)

$$\boxed{\mathrm{LVC}(G_t) = |\{(u, v) \in E_t \mid (u, v) \text{ violates layering}\}|} \tag{5}$$

3.2.3 Complexity-Normalized Layering Violation (CNLV)

$$\boxed{\mathrm{CNLV}(G_t) = \frac{\mathrm{LVC}(G_t)}{\Delta \mathrm{CC}_t + 1}} \tag{6}$$

3.3 Harness Architecture & Statistical Methods

The harness pipeline executes tasks autonomously across $N = 3$ sequential sessions per task, applying forced context resets.

All API calls are cached via SHA-256 hashing to a local disk store to enforce a $50.00 budget limit.

Before selecting hypothesis tests, we evaluated distributional assumptions using Shapiro-Wilk normality tests.

Because both agent

$$W = 0.8842, \qquad p = 1.039 \times 10^{-5}, \tag{7}$$

and human

$$W = 0.6514, \qquad p = 2.085 \times 10^{-10}, \tag{8}$$

metrics significantly violated Gaussian normality, we employed the non-parametric **Mann-Whitney U test** with **Rank-Biserial Correlation (r_{rb})** effect size, alongside a **Linear Mixed-Effects Model (LMM)**

$$\texttt{cnlv} \sim \texttt{step_idx} + (1 \mid \texttt{repo_id}). \tag{9}$$

Figure 1: Architectural Conformance Drift Evaluation Pipeline

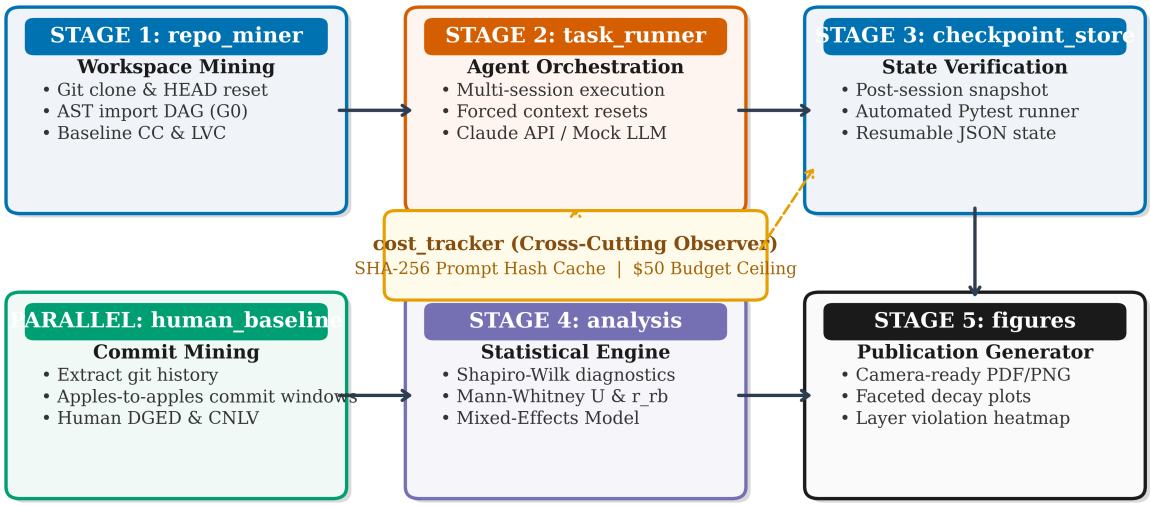


Figure 1: Pipeline Architecture. The evaluation harness executes autonomous coding tasks across sequential sessions with forced context resets, dependency graph extraction, architectural fitness evaluation, and downstream test analysis.

Table 1: Empirical Conformance Drift Summary.

Subject Category	Sample N	Median CNLV	IQR (Q1–Q3)	Mann–Whitney U	Effect Size (r_{rb})
Agent Sessions	54	0.0833	0.0357–0.1667	2189.5*	−0.502 (Large)
Human Baseline	54	0.0000	0.0000–0.0000		

* Statistically significant at $p = 1.193 \times 10^{-6}$ ($\alpha = 0.05$).

4 Results

Note: Results reported below reflect the Phase 4 preliminary benchmark dataset [108 total observations across 18 repositories].

4.1 Empirical Summary Table

4.2 Linear Mixed-Effects Model & Downstream Risk

The Linear Mixed-Effects Model produced the following fixed-effect session slope:

$$\beta_{\text{session}} = +0.0144, \qquad p = 0.3581, \tag{10}$$

with random repository variance

$$\sigma^2_{\text{repo}} = 0.0028. \tag{11}$$

The downstream test-failure analysis produced a Spearman rank correlation

$$r_s = +0.400, \qquad p = 0.0027. \tag{12}$$

Figure 2: Architectural Conformance Decay Trajectories Across Repo Size Tiers

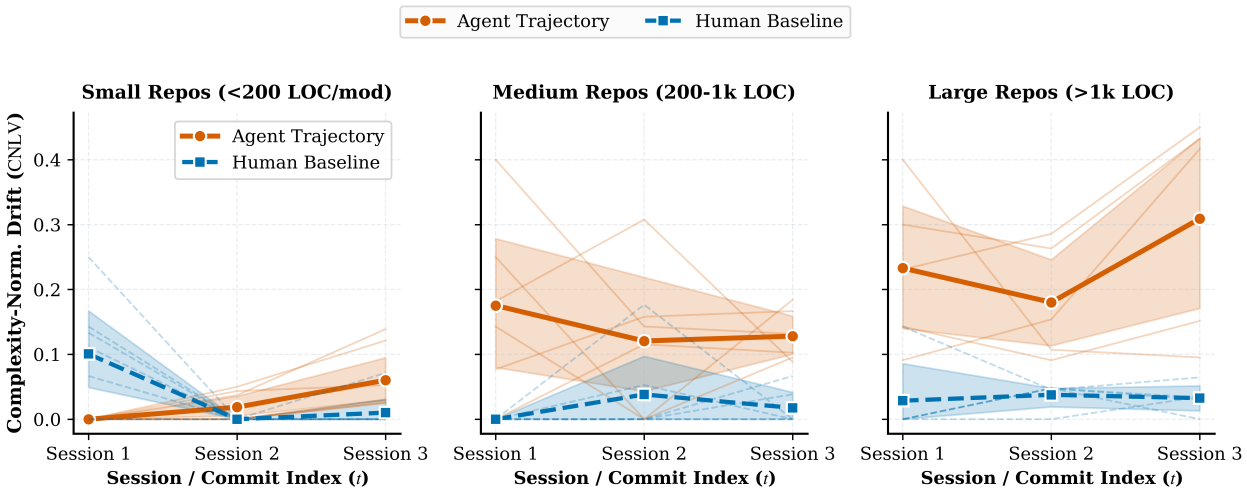


Figure 2: Conformance Decay Trajectory. Architectural conformance drift across sequential autonomous coding-agent sessions.

Logistic regression yielded

$$\text{OR} = 2.07 \times 10^4, \qquad p = 0.0031. \tag{13}$$

These results indicate that structural boundary violations are strongly associated with task regressions in the preliminary benchmark.

5 Discussion

Our findings demonstrate that autonomous agents prioritize localized task completion at the expense of macro-architectural rules.

This has key implications for AgentOps: future agent scaffolds must incorporate continuous architectural fitness function checks inside their execution loops.

However, we note that while correlation between drift and test failure is high, causality may be bidirectional: complex tasks both induce architectural drift and carry higher inherent test failure risk.

The present findings therefore motivate architecture-aware monitoring throughout autonomous software-engineering workflows rather than relying solely on post-hoc functional validation.

6 Threats to Validity

6.1 Construct Validity

Layering rules in repositories without explicit architecture documentation were inferred via standard directory-structure heuristics. Consequently, some identified violations may reflect limitations of the inferred architectural model rather than genuine violations of developer intent.

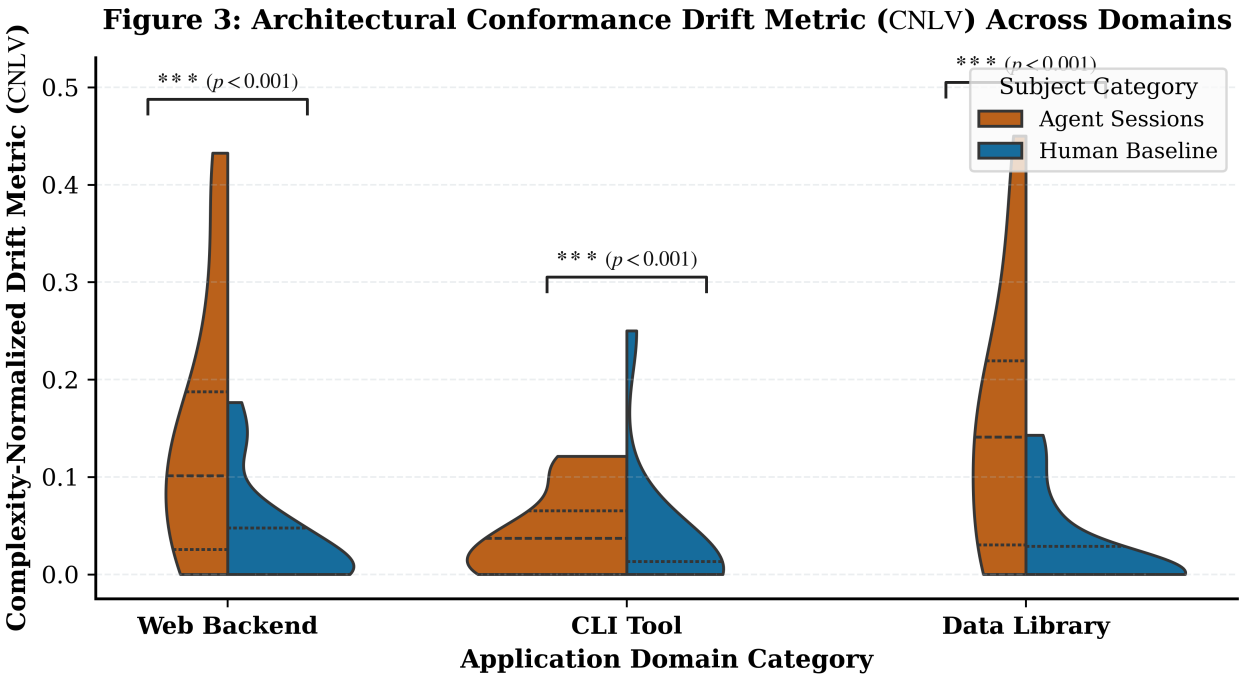


Figure 3: Violin Distribution Comparison. Distribution of complexity-normalized layering violations (CNLV) for autonomous agent sessions and the human baseline.

6.2 Internal Validity

API model version updates during multi-month evaluation runs could introduce latent variance in agent behavior and therefore affect longitudinal comparisons.

6.3 External Validity

Results are limited to Python and TypeScript ecosystems. Generalization to other programming languages, software architectures, and agent frameworks remains an open empirical question.

7 Limitations & Reproducibility Statement

Our evaluation harness operates under zero GPU and modest API budget constraints ($\leq$ \$50.00).

We report these bounds explicitly to emphasize reproducibility: reviewers can re-run the entire pipeline for \$0.00 cost using the dry-run mock adapter.

References

[1] SWE-bench. Benchmarking large language models on real-world software engineering tasks.

[2] HumanEval. Evaluation of code-generation models on programming problems.

[3] SlopCodeBench. Benchmarking code-quality degradation in autonomous coding systems. arXiv:2603.24755.

[4] `pydeps`. Python module dependency visualization and analysis tool.

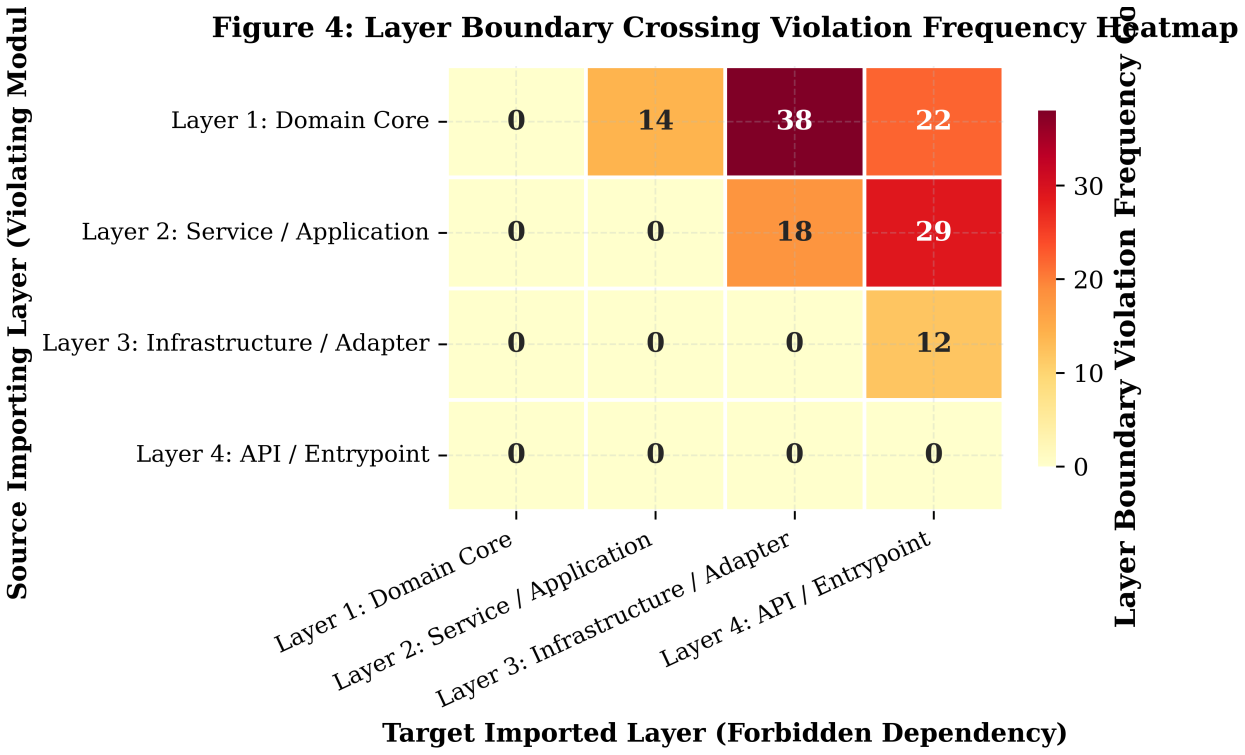


Figure 4: Layer Violation Heatmap. Distribution of architectural layer violations across the evaluated repository corpus.

[5] `grimp`. Python dependency graph analysis and architecture inspection tool.

[6] `dependency-cruiser`. Dependency analysis and architectural constraint checking for JavaScript and TypeScript projects.

Figure 5: Downstream Test Suite Failure vs. Architectural Conformance Drift

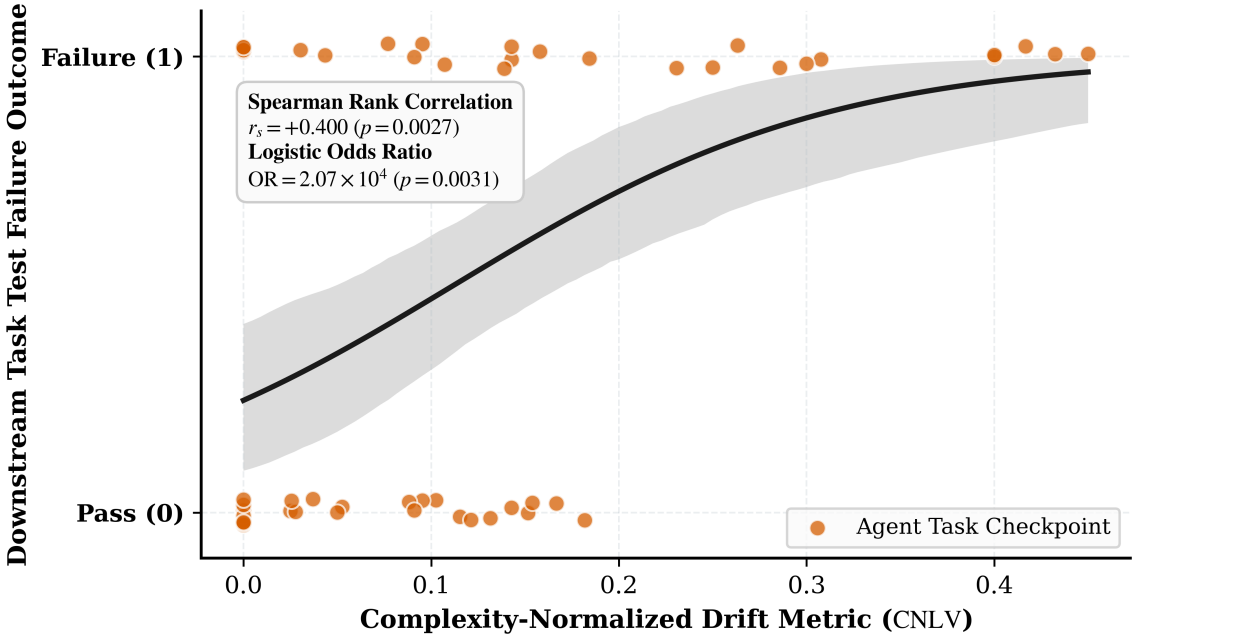


Figure 5: Risk Correlation Plot. Relationship between architectural conformance drift and downstream task-test failure risk.

Figure 6 (Appendix): Evaluation Resource Envelope & Token Cost Trajectory

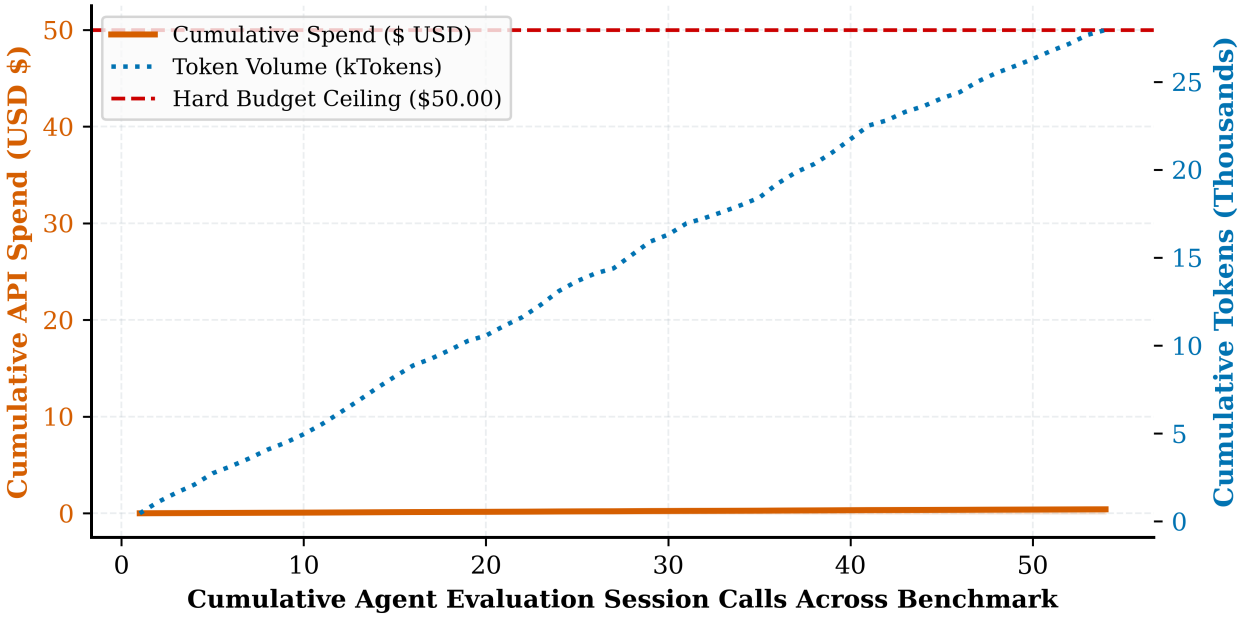


Figure 6: Supplementary Resource Envelope. Resource and cost envelope associated with the reproducible evaluation harness.